How the internet works ?

protocol → https://www.scrimba.com

- www → subdomain
- scrimba → domain
- .com → TLD (Top-Level Domain) { .org, .net, etc.

IP Address : a unique numerical label assigned to each device on a network to identify & enable communication b/w devices.

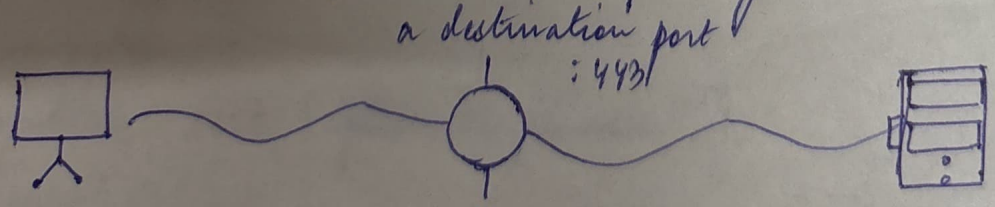
198.51.100.25

"Resolving the domain"
DNS

Port : a no. (0 - 65535) that identifies a specific process or service on a computer

Common ports →
- 80 → insecure web (HTTP)
- 443 → secure web (HTTPS)
- 25/587/465 → Email
- 22 → SSH (secure remote login)

0 - 1023 for standard services (e.g. 443 for secure web)
1024 - 49151 for DBs & servers (e.g. 3306 for MySql)
(49152 - 65535) ephemeral / temporary



Domain Name System (DNS):

CLIENT → local cache → Recursive Resolver (ISP)

- ↙ cache
- ↘ root server
  - ↓ who handles .com?
  - TLD server
    - ↓ who handles scrimba.com?
    - Authoritative server
      - ↓ IP address for scrimba.com?
      - result.

How Devices Talk

A protocol is a set of rules that define how data is formatted, transmitted & received b/w devices on a network.

IP – Internet Protocol → A set of rules for addressing, routing & delivering packets.

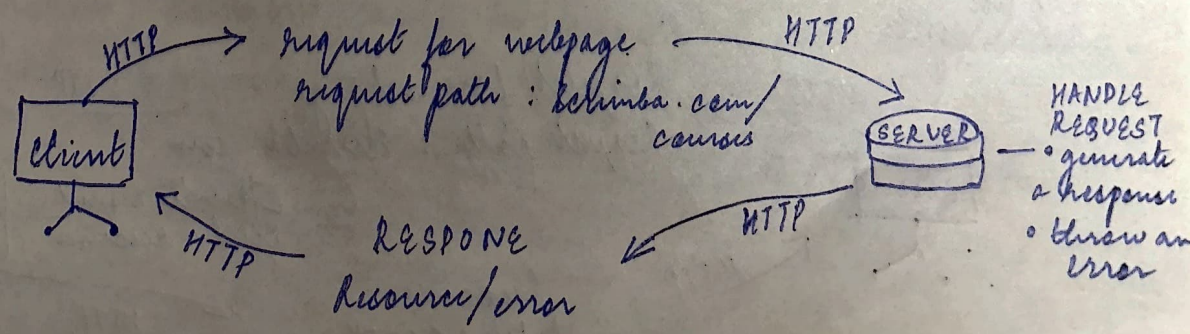
TCP – Transmission Control Protocol → A protocol which makes sure no packets are lost & everything is in the right order.

Port → Socket → TCP handshake → data sent in packets
[443] [IP + Port] [Set the rules] [packets are numbered & checked]

UDP – User Datagram Protocol → Also works over IP, but while TCP is thorough & reliable, UDP is fast, unreliable & makes the best effort to deliver.

Application protocols →
- HTTP/HTTPS
- FTP/SFTP - file transfer
- SMTP/IMAP - email

HTTP:



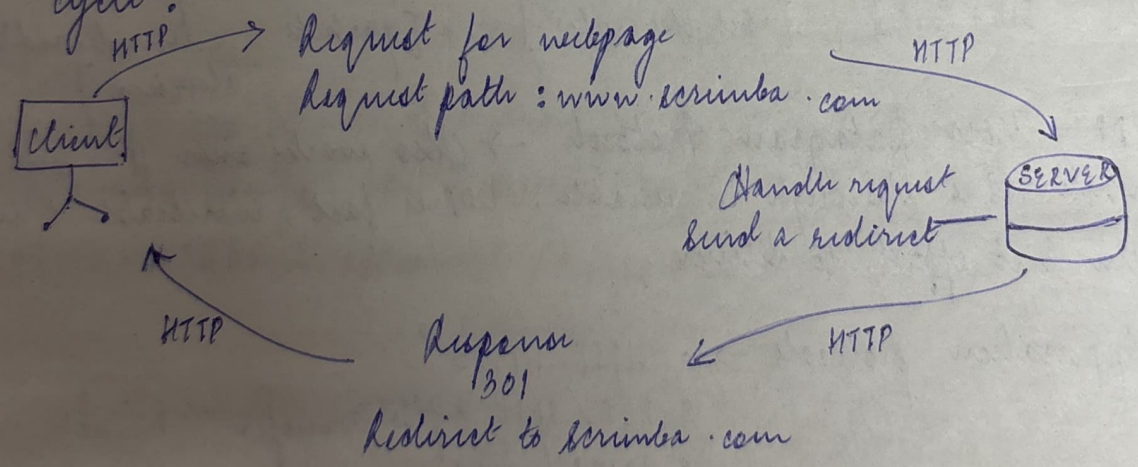
By default HTTP is stateless, which means the server doesn't automatically remember you b/w requests.
To keep track of things like logins or shopping carts, the browser & server use helpers like cookies, tokens or sessions.

HTTPS : encrypted w/ TLS (Transport Layer Security)
→ Certificates & trusts
→ The handshake b/w browser & server
→ Secure communication

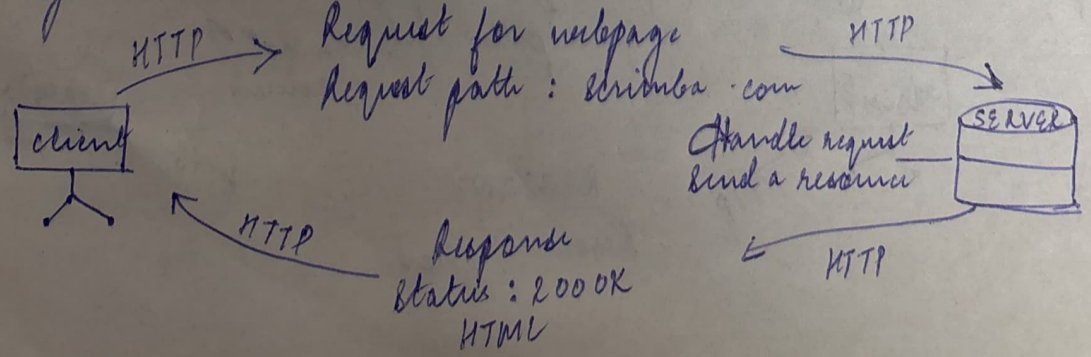
1) Use valid certificates
2) Keep encryption settings up to date
3) Make sure users get directed to HTTPS

Dev Tools :-
Network : Status codes —
- 200 OK
- 301 Permanent Redirect
- 404 Not found
- 500 Internal server error

1st cycle :



2nd cycle :



Hosting : Your website / application needs a place to live where people can find it.

1) Shared hosting : Many websites are placed on the same machine & share its resources. It's cheap & easy to use but slower & less flexible - you have less control.

2) VPS (virtual Private Server) : A single physical server divided into multiple virtual servers. This gives you more control & better performance w/o paying for a whole physical server.

3) Dedicated Server : One customer gets an entire machine to themselves. You get max^m. power, performance, and control but also higher costs & more responsibility for managing it.

4) Cloud hosting : Your site runs on a pool of servers. This makes it easy to scale up or down depending on demand & adds reliability, because if one server fails, other takes over.

Hosting services :
- Domain registration
- TLS/SSL certificate automation (so you can enable HTTPS easily)
- Monitoring tools (to check uptime & performance)
- Security features like firewalls, DDOS protection & backups.

CLI Network Tools :-

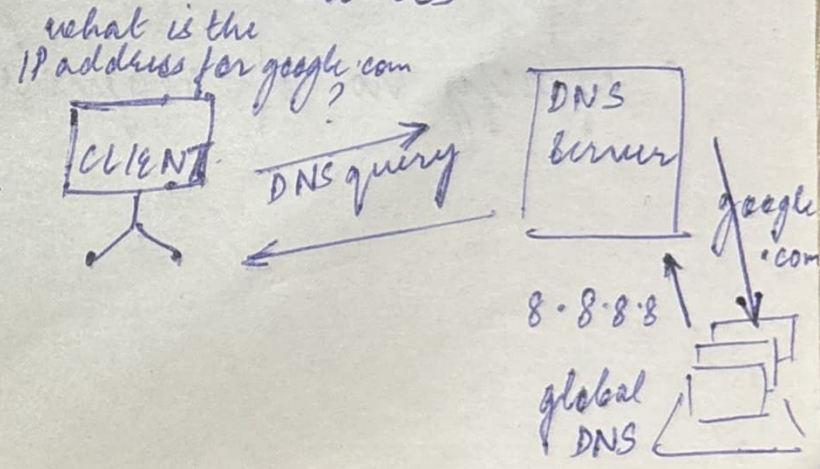
Resolve DNS Records :

Look up an IP address :

- Protocol : Domain Name System (DNS)
- we provide a domain name & get back the IP address associated w/ the domain.
- If DNS is broken, domains can't be translated into IP addresses, i.e. we cannot access web pages via their domain names.

How it works :

- Command (all OS's) : nslookup <domain>
- Example : nslookup google.com



Output of nslookup:

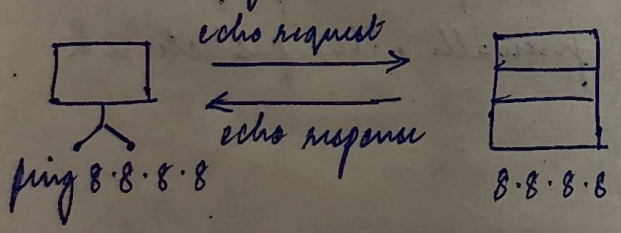
- DNS server info:
  - Bash/zsh: IP address & IP address + port
  - windows: Domain & IP address
- Answer, i.e. the result of our query:
  - "Name" = domain name
  - "Address" = IP address
  - Either "authoritative"/"non-authoritative"

Test Network Connectivity

- Protocol: Internet Control Message Protocol (ICMP)
- ICMP is only testing the connectivity itself.
- we can't use it to request data or contents from web pages.

Ping an IP address

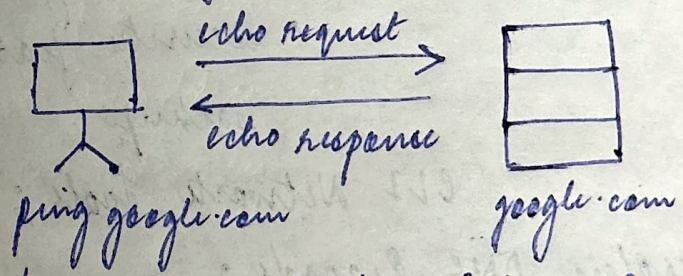
- Command: ping <IP address>
- Ex: ping 8.8.8.8



Test: connectivity & latency

Ping a domain

- Command: ping <domain>
- Ex: ping google.com.

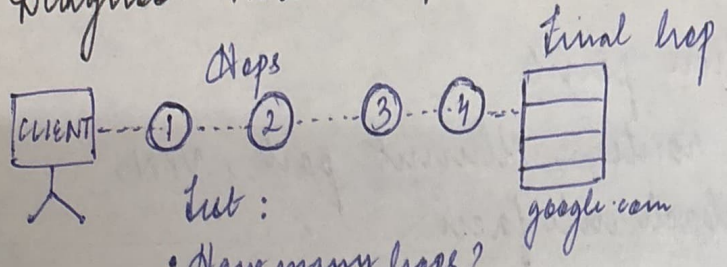


Test: Connectivity, latency & DNS resolution.

Output: 1) echo request
2) echo response
3) statistics, all packets

- The process of sending off echo requests & getting back echo responses runs continuously until we stop it.
- Specify no. of requests: ping google.com -c <N>

Diagnose Network Paths :



Test :

- How many hops ?
- What is the Round-trip time ?

- Command : traceroute < domain >

Get Domain Registration Info :

1) Domain name / Domain ID
2) Creation / updation / Expiry Date
3) DNS server (s)
4) Domain owner

Use cases :-

- Administration
- Investigation
- Troubleshooting

- Command : whois < domain >
- Ex : whois google.com

Fetch HTTP headers :

- HTTP headers contain metadata about requests / responses — not about the page itself. For example :
  - HTTP version - inspect server behavior
  - Status code - check reachability
  - date of server response
  - content type
  - cookies / cache rules : debug & trouble shoot
- Command : curl -I https:// < domain >
- Ex : curl -I https://google.com

NOTE : always specify HTTPS before domain.

→ request is sent over encrypted TLS connection

→ if no HTTP header in response = server does not support HTTPS / TLS.

Show Network Interfaces:

Network interfaces : connection point
eg: wifi router, ethernet port, VPN, loopback interfaces

- Command : ifconfig

Output : For each interface, we get information about eg:
- Interface name (assigned dynamically by OS)
- Network adapter (MAC address)
- Local/private IP addresses (IPv4/IPv6)
- Network status (UP/DOWN)

OS specific output :
Network status : "status : active" v/s "status : inactive"
IP addresses : "inet" = IPv4 / "inet6" = IPv6

Use cases :
- Inspect network interfaces
- Diagnose connectivity issues :
  - Is an interface up or down?
  - Does it have an IP address?
  - Is it configured correctly?

Test Server Availability:

Server availability via TCP port.

- Protocol : TCP
- HTTPS connection : port 443
- If the TCP port is open & traffic can reach it, then the server is online & available.
- Command : nc -vz <domain> <port>
- Eg : nc -vz google.com 443

Output : → connection succeeded
→ {Failure : connection refused / Host unreachable / package dropped silently.
CTRL+C}

Inspect TLS Handshake :

- Command : openssl s_client -connect <domain> : 443
  Secure sockets layer
- Ex : openssl s_client -connect example.com : 443
- Output : certificate & certificate chain
  Protocol version & cipher suite
  verification of the certificate